

UFUG 2602

高级排序

Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

IoT, DSA, INTR Thrusts
香港科技大学 (广州)

大纲

- **主定理**
- 快速排序
- 排序下界

主定理：切蛋糕的故事

假设你有一块大蛋糕要和朋友分享，但太大无法一次切完。可以这样划分：

- 1.切蛋糕： 每次把蛋糕切成大小大致相等的 α 块。
- 2.处理每一块： 对每一块重复切割，直到小到可以分给朋友。
- 3.额外工作： 每一轮切割都有额外任务，例如洗刀或擦桌子。这需要 $g(n)$ 时间，其中 n 是当前蛋糕的大小。

现在的问题是：整个切蛋糕过程需要多长时间？



主定理

设 $f(n)$ 是对每个整数 $n > 0$ 都返回正值的函数。已知：

$$\begin{aligned} f(1) &= O(1) \\ f(n) &\leq \alpha \cdot f\left(\left\lceil \frac{n}{\beta} \right\rceil\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2) \end{aligned}$$

其中 $\alpha \geq 1, \beta > 1, \gamma \geq 0$, 且 $\lambda \geq 0$ 为常数。于是：

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

α : 每次将蛋糕分成的份数。

β : 与原蛋糕相比, 每块大小缩小的倍数。

$g(n)$: 每次划分所需的额外工作量。

该定理可通过仔细应用前面见过的“展开法”来证明。细节繁琐, 此处从略。

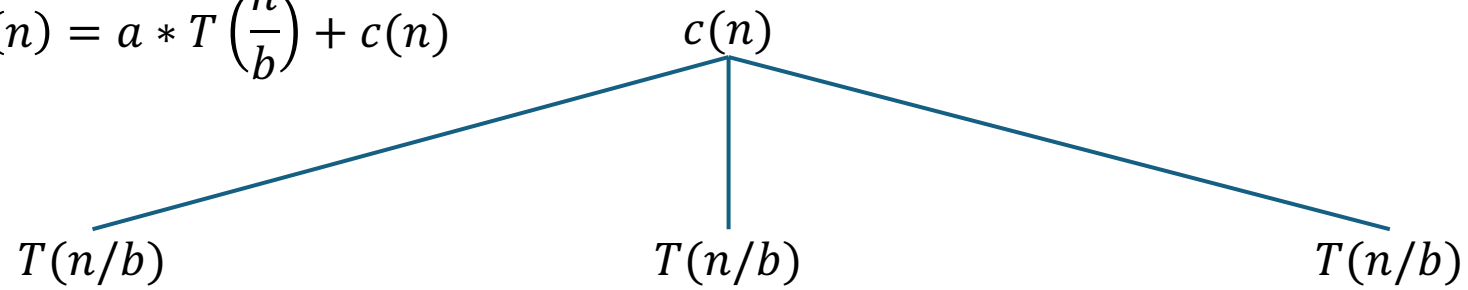
主定理

$$T(n) = b * T\left(\frac{n}{b}\right) + c(n)$$

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

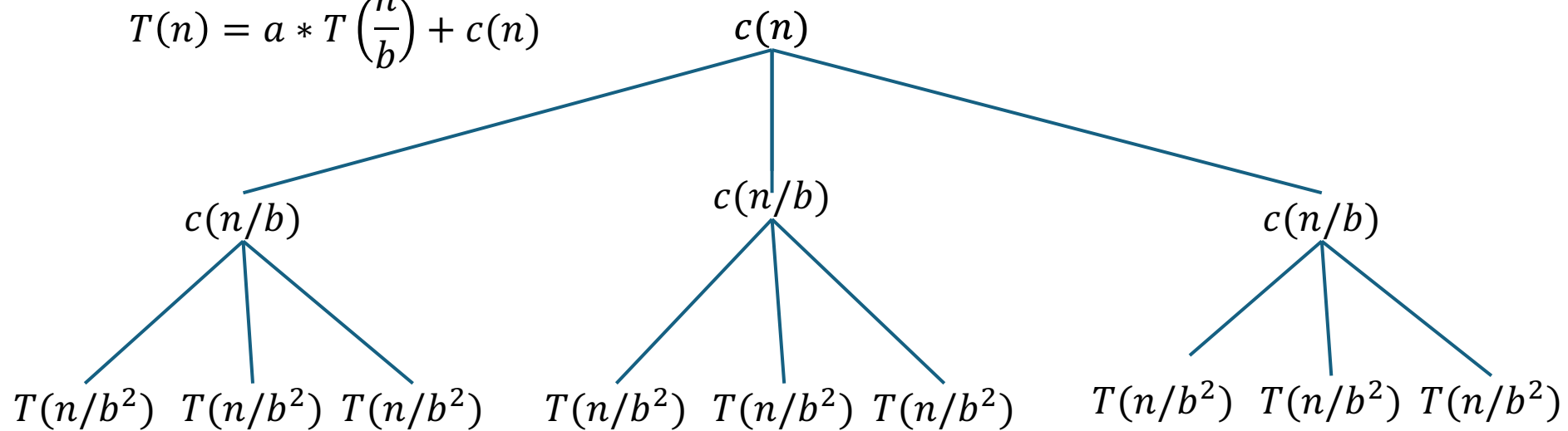
主定理

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



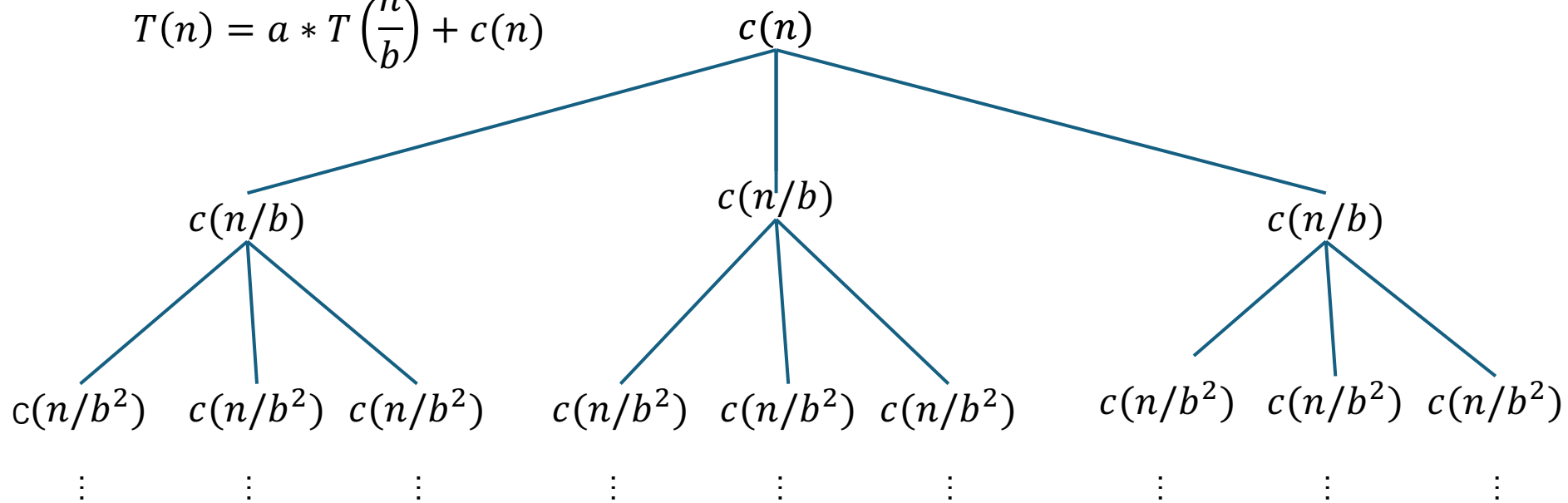
主定理

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



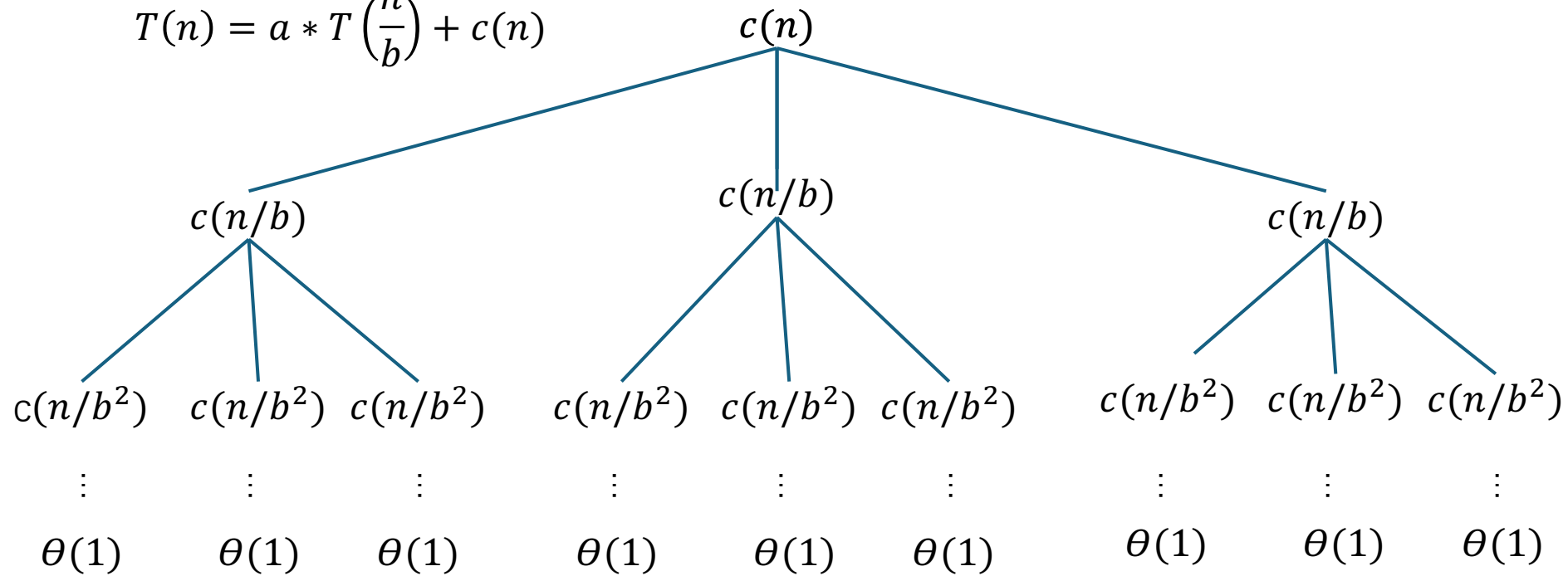
主定理

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



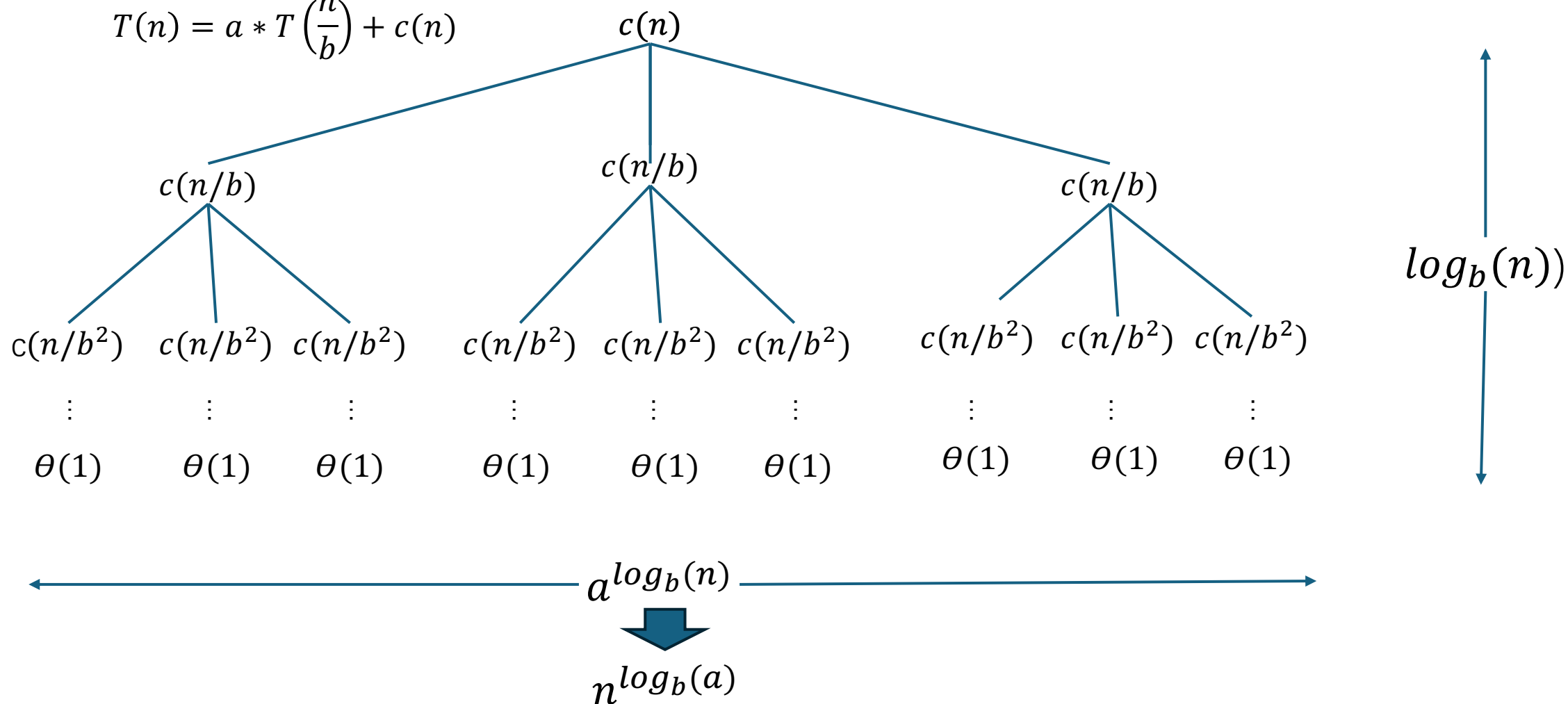
主定理

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

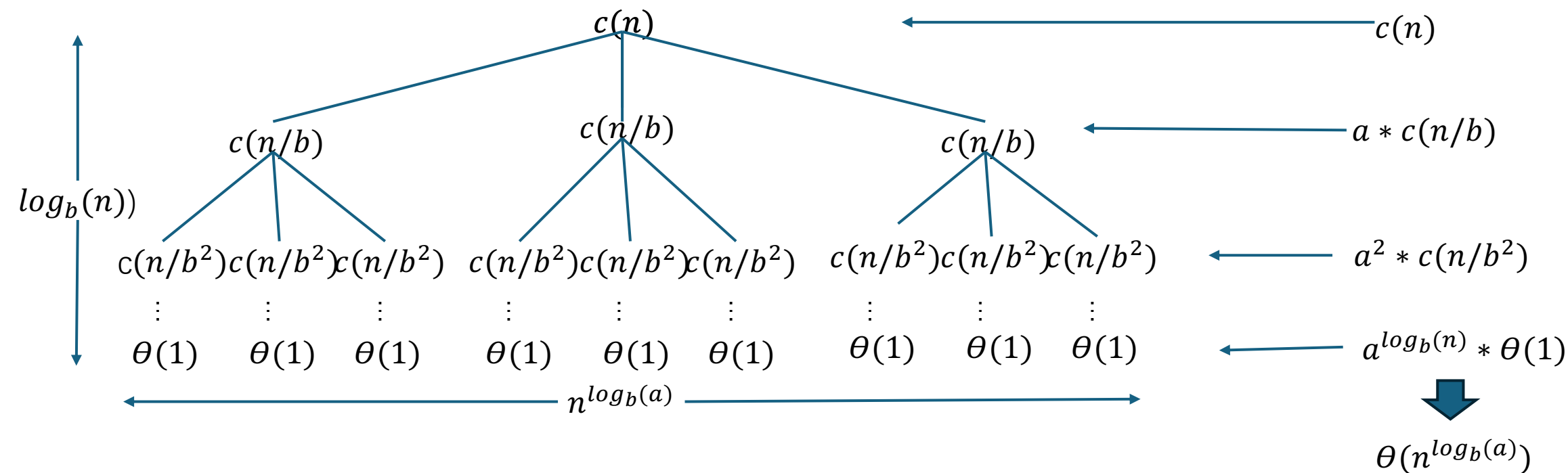


主定理

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

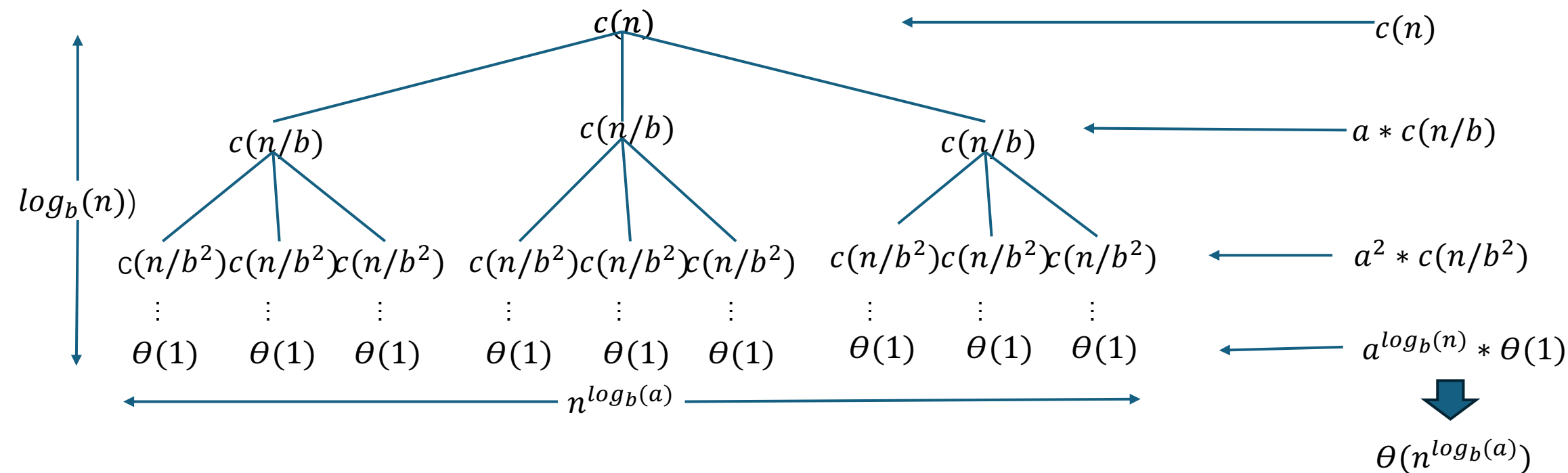


主定理



$$T(n) = c(n) + a \cdot c\left(\frac{n}{b}\right) + a^2 \cdot c\left(\frac{n}{b^2}\right) + \dots \theta(n^{\log_b(a)})$$

主定理



$$T(n) = \sum_{j=0}^{\log_b(n)-1} a^j * c(n/b^j) + \theta(n^{\log_b(a)})$$

主定理

$$T(n) = \theta(n^{\log_b(a)}) + \sum_{j=0}^{\log_b(n-1)} a^j * c(n/b^j)$$

1. 若问题变得更容易 -> 根最难
2. 若难度均匀分布 -> 各层都相关

$$T(n) = \theta(C(n))$$

$$T(n) = height * (some\ level)$$

$$T(n) = \log_b(n) * \theta(n^{\log_b(a)}) = \theta(n^{\log_b(a)} \log_b(n))$$

3. 若问题变得更难 -> 叶子最难

$$T(n) = \theta(n^{\log_b(a)})$$

主定理

设 $f(n)$ 是对每个整数 $n > 0$ 都返回正值的函数。已知：

$$\begin{aligned} f(1) &= O(1) \\ f(n) &\leq \alpha \cdot f\left(\left\lfloor \frac{n}{\beta} \right\rfloor\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2) \end{aligned}$$

其中 $\alpha \geq 1, \beta > 1, \gamma \geq 0$, 且 $\lambda \geq 0$ 为常数。于是：

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

α : 每次将蛋糕分成的份数。

β : 与原蛋糕相比, 每块大小缩小的倍数。

$g(n)$: 每次划分所需的额外工作量。

该定理可通过仔细应用前面见过的“展开法”来证明。细节繁琐, 此处从略。

主定理

示例 1:

考虑二分查找的递推式:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq f(\lceil n/2 \rceil) + c_2 \quad (\text{for } n \geq 2) \end{aligned}$$

因此, $\alpha = 1, \beta = 2, \gamma = 0$, 且 $\lambda = 0$ 。由于 $\log_\beta \alpha = \gamma$, 可知 $f(n) = O(n^0 \cdot \log^{0+1} n) = O(\log n)$

主定理

示例 2:

考虑归并排序的递推式:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq 2 \cdot f(\lceil n/2 \rceil) + c_2 n \quad (\text{for } n \geq 2) \end{aligned}$$

因此, $\alpha = 2, \beta = 2, \gamma = 1$, 且 $\lambda = 0$ 。由于 $\log_\beta \alpha = \gamma$, 可知
 $f(n) = O(n^\gamma \cdot \log^{0+1} n) = O(n \log n)$.

主定理

示例 3:

考虑递推式:

$$f(1) \leq c_1$$

$$f(n) \leq 2 \cdot f(\lceil n/4 \rceil) + c_2 \sqrt{n} \cdot \log_2 n \quad (\text{for } n \geq 2)$$

因此, $\alpha = 2, \beta = 4, \gamma = 1/2$, 且 $\lambda = 0$ 。由于 $\log_\beta \alpha = \gamma$, 可知

$$f(n) = O(n^\gamma \cdot \log^{\lambda+1} n) = O(\sqrt{n} \log^2 n).$$

。

主定理

示例 4:

考虑递推式:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq 2 \cdot f(\lceil n/2 \rceil) + c_2 \sqrt{n} \quad (\text{for } n \geq 2) \end{aligned}$$

因此, $\alpha = 2, \beta = 2, \gamma = 1/2$, 且 $\lambda = 0$ 。由于 $\log_{\beta} \alpha > \gamma$, 可知 $f(n) = O(n^{\log_{\beta} \alpha}) = O(n)$

主定理

示例 5:

考虑递推式:

$$f(1) \leq c_1$$

$$f(n) \leq 13 \cdot f(\lceil n/7 \rceil) + c_2 n^2 \quad (\text{for } n \geq 2)$$

因此, $\alpha = 13, \beta = 7, \gamma = 2$, 且 $\lambda = 0$ 。由于 $\log_\beta \alpha < \gamma$, 可知 $f(n) = O(n^\gamma \log^\lambda n) = O(n^2)$

Q1. 应用主定理

- 对于递推式：

$$T(n) = 4T(n/2) + n$$

- **问题：**
用主定理确定时间复杂度。

Q2. 主定理属于哪一种情形?

- 对于递推式:

$$T(n) = 2T(n/2) + n \log n$$

- **问题:**
- 它属于哪一种情形?
- 最终复杂度是什么?

Q3. 判断正误

- 对于递推式：

$$T(n) = 8T(n/2) + n^2$$

- 陈述：“时间复杂度为 $\Theta(n^2)$ 。”

Q4. 把故事写成递推式

- 某算法：
 - 将问题分成 3 个规模相等的子问题
 - 每个子问题的规模为 $n/4$
 - 花费 n 额外工作进行合并
- Question:
 - 写出递推式
 - 用主定理求解

Q5. 主定理习题

- 对于递推式：

$$T(n) = 5T(n/2) + n^2$$

$$T(n) = 2T(n/4) + \sqrt{n}$$

- 问题：

- 它属于哪一种情形？
- 最终复杂度是什么？

大纲

- 主定理
- **快速排序**
- 排序下界

快速排序：整理书架



1.选择枢轴：随机选一本书作为枢轴（例如一本中等高度的书）。

2.划分：把书分成两组：

1. 左组：比枢轴矮的书。

2. 右组：比枢轴高的书。

枢轴留在原位，其最终位置现已确定。

3.递归重复：

1. 对左组选取新枢轴并继续划分。

2. 对右组选取新枢轴并继续划分。

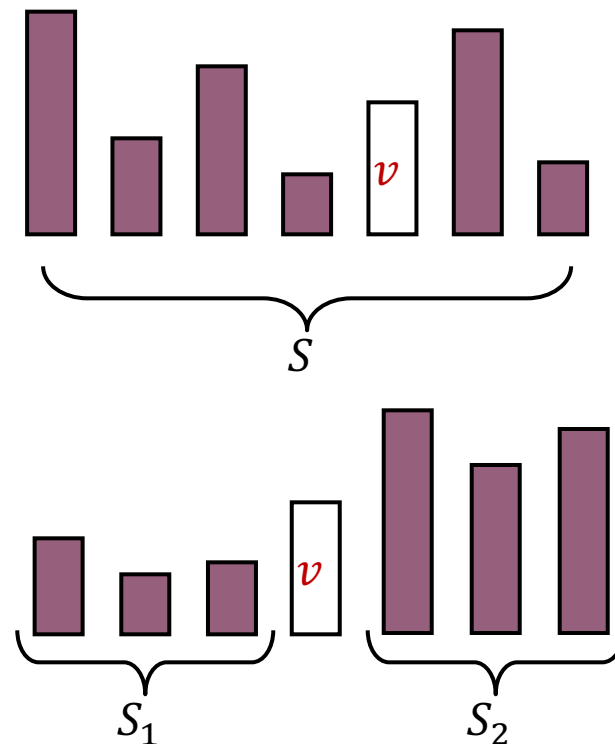
3. 重复直到每组只剩一本或没有书。

4.合并：按顺序摆放：左组 + 枢轴 + 右组。

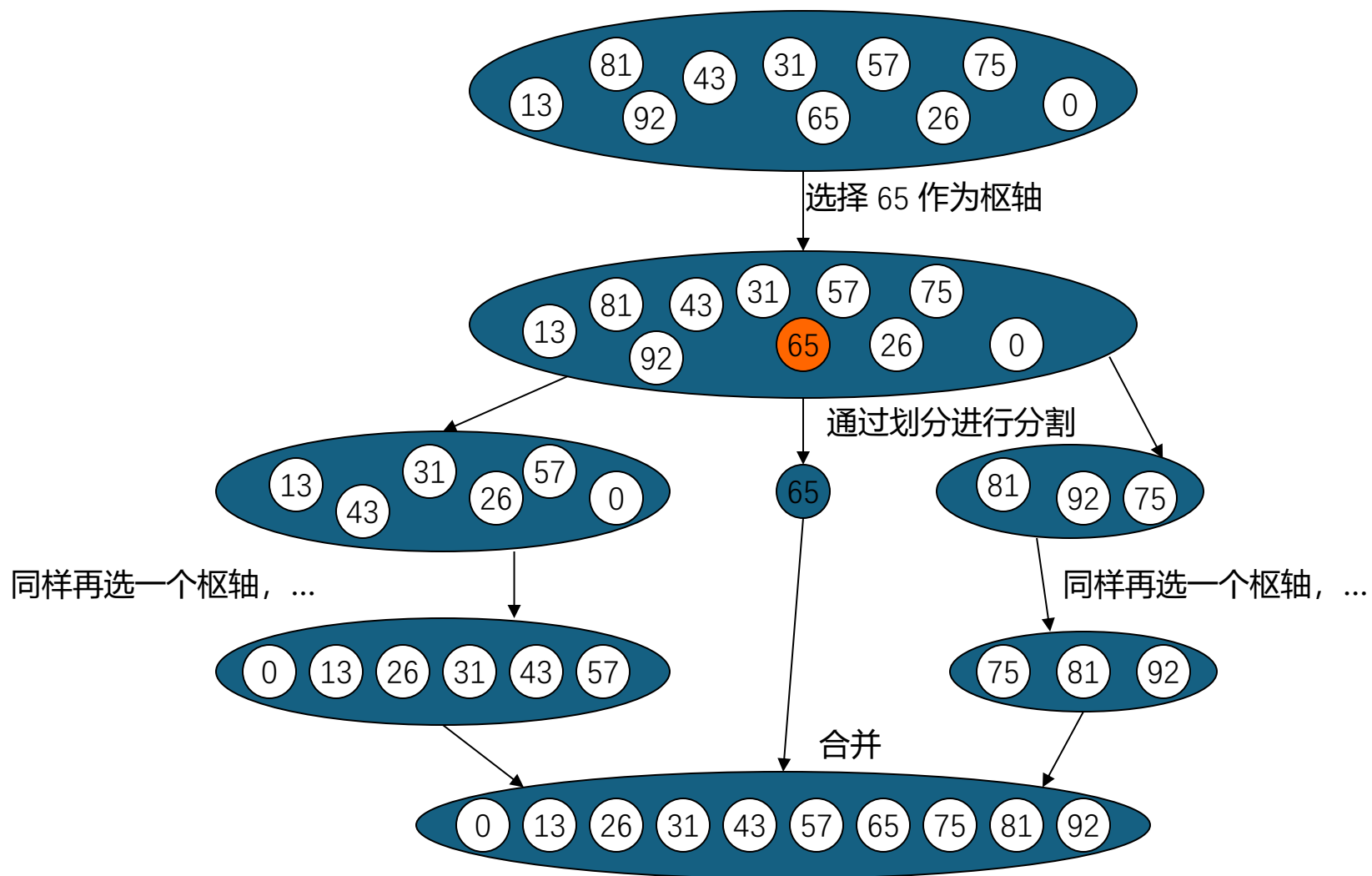


快速排序：分治

- 分治。
 - 假设所有数字互不相同。
- Divide step:
 - 将集合 S 分成两个子集 S_1 和 S_2 。
 - 任选一个数 v 作为 S 中的枢轴。
 - 将 $S \setminus \{v\}$ 划分成两个集合：
 - $S_1 = \{x \in S \setminus \{v\} \mid x < v\}$ and $S_2 = \{x \in S \setminus \{v\} \mid x > v\}$.
- 解决步：递归排序 S_1 和 S_2 。
- 合并步：将 S 排成已排序的 S_1 后接 v ，再接已排序的 S_2 。
 - 这一步几乎无需做什么。



快速排序示例



Q6：快速排序过程追踪

仅对以下数组执行一次划分：

[9, 4, 7, 3, 8, 2]

以第一个元素作为枢轴。

快速排序：分治

- 伪代码。

QuickSort(a, left, right):

if left < right:

在 a[left..right] 中选择枢轴 v

将 a[i] < v 移到左边 (a[left..p-1])

将 a[i] ≥ v 移到右边 (a[p+1..right])

a[p] = v /* a[p] 保存枢轴值 v */

quickSort(a, left, p-1)

quickSort(a, p+1, right)

划分



- 问题 1：如何高效完成划分？
- 问题 2：如何选择枢轴？

快速排序：划分

- 假设用额外数组 $b[]$ 对长度为 n 的 $a[]$ 做划分。
- 设枢轴值为 p 。
- 令 $i = 0, j = n-1$ 。
- 从 $k=0$ 扫描 $a[k]$ 直到 $n-1$ 。
 - 若 $a[k] < p$, 则从左侧放入 $b[i++]$ 。
 - 若 $a[k] > p$, 则从右侧放入 $b[j--]$ 。
- 最后把 p 放入剩余位置。

快速排序：划分

- 假设用额外数组 $b[]$ 对长度为 n 的 $a[]$ 做划分。
- 设枢轴值为 p 。
- 令 $i = 0, j = n-1$ 。
- 从 $k=0$ 扫描 $a[k]$ 直到 $n-1$:
 - 若 $a[k] < p$, 则从左侧放入 $b[i++]$;
 - 否则从右侧放入 $b[j--]$ 。
- 最后把 p 放入剩余位置。

- 划分时间为 $O(n)$
- 问题是什么？需要额外空间 $b[]$ 。

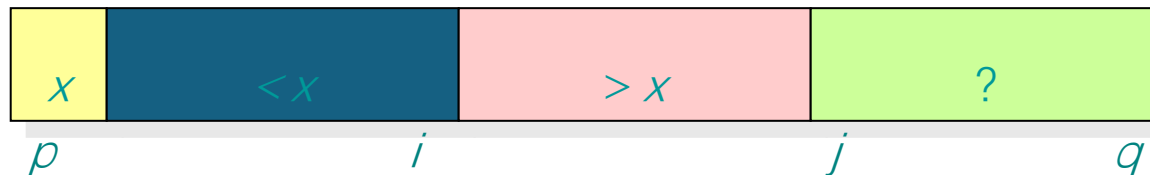
更节省空间的划分

不失一般性，假设第一个元素为枢轴。

```
Partition(A,p,q): #A[p..q]
   $x \leftarrow A[p]$       #pivot = A[p]
   $i \leftarrow p$ 
  for  $j \leftarrow p + 1$  to  $q$ 
    do if  $A[j] < x$ 
      then  $i \leftarrow i + 1$ 
      exchange  $A[i] \leftrightarrow A[j]$ 
  exchange  $A[p] \leftrightarrow A[i]$ 
  return  $i$ 
```

- 运行时间 = $O(n)$ (n 个元素)。
- 额外空间为常数

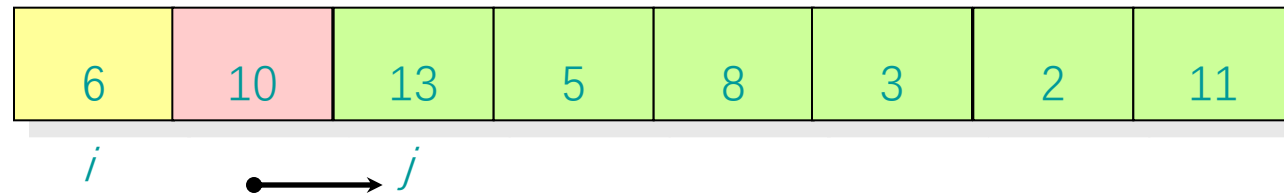
不变式:



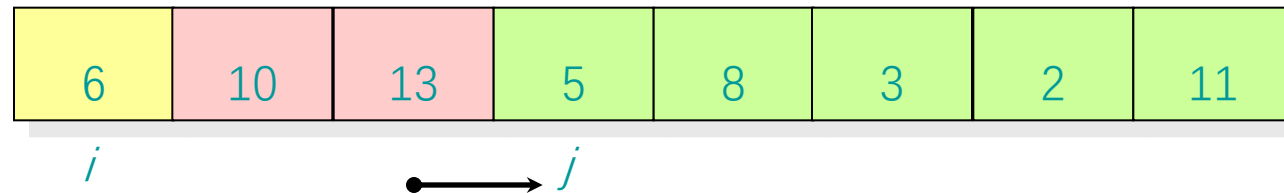
划分示例

6	10	13	5	8	3	2	11
i	j						

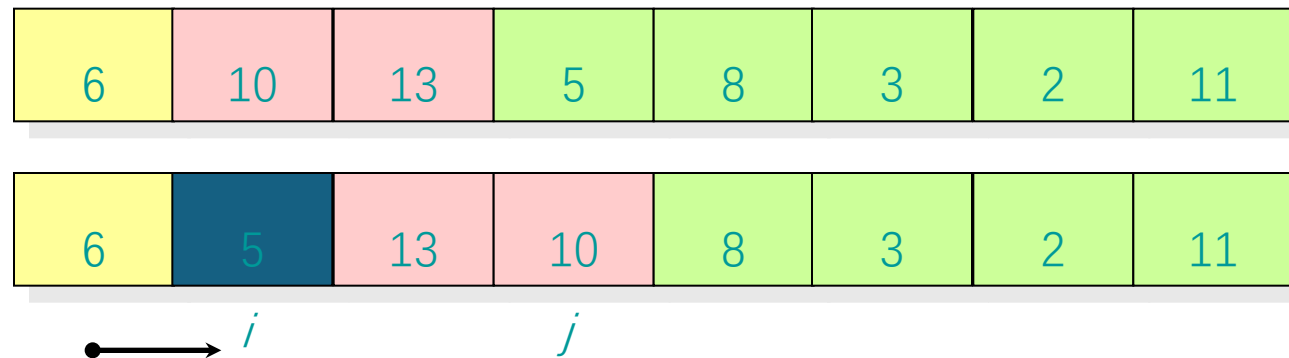
划分示例



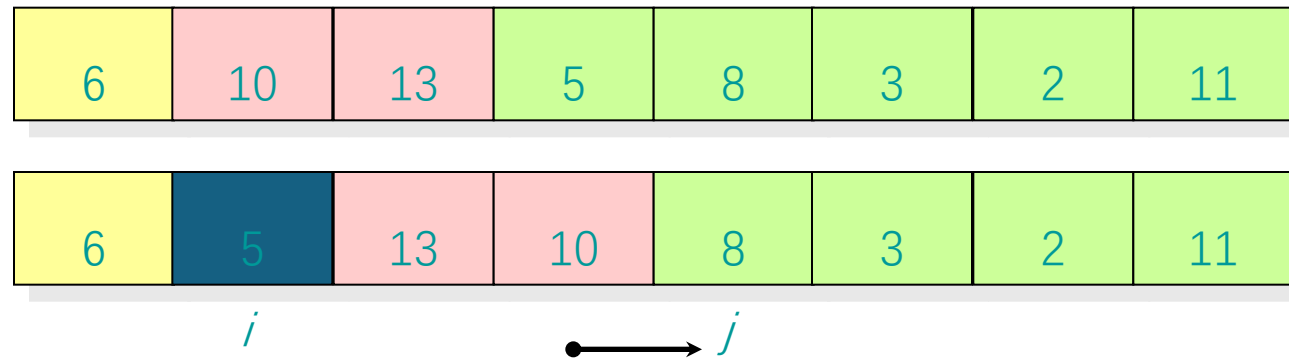
划分示例



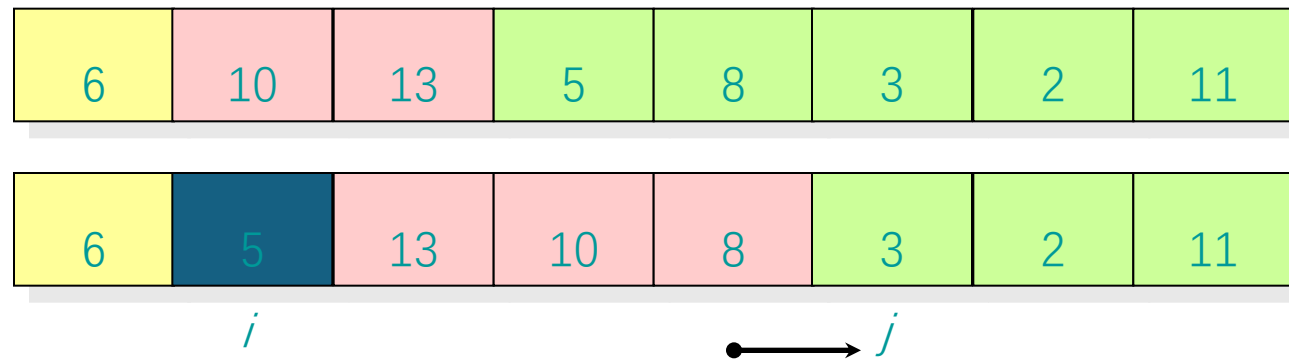
划分示例



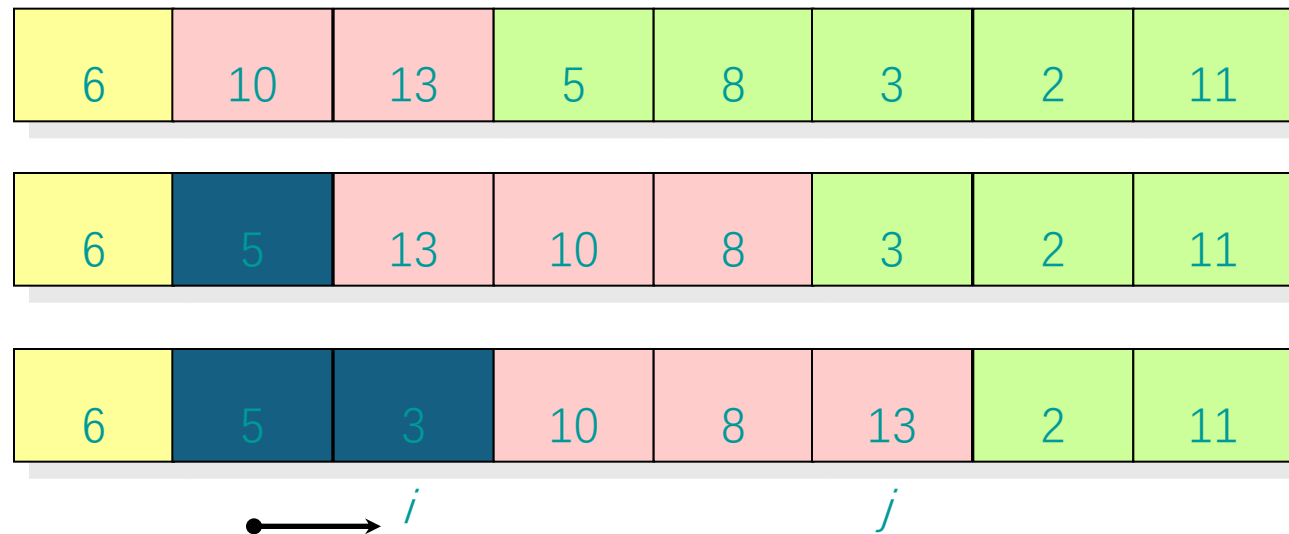
划分示例



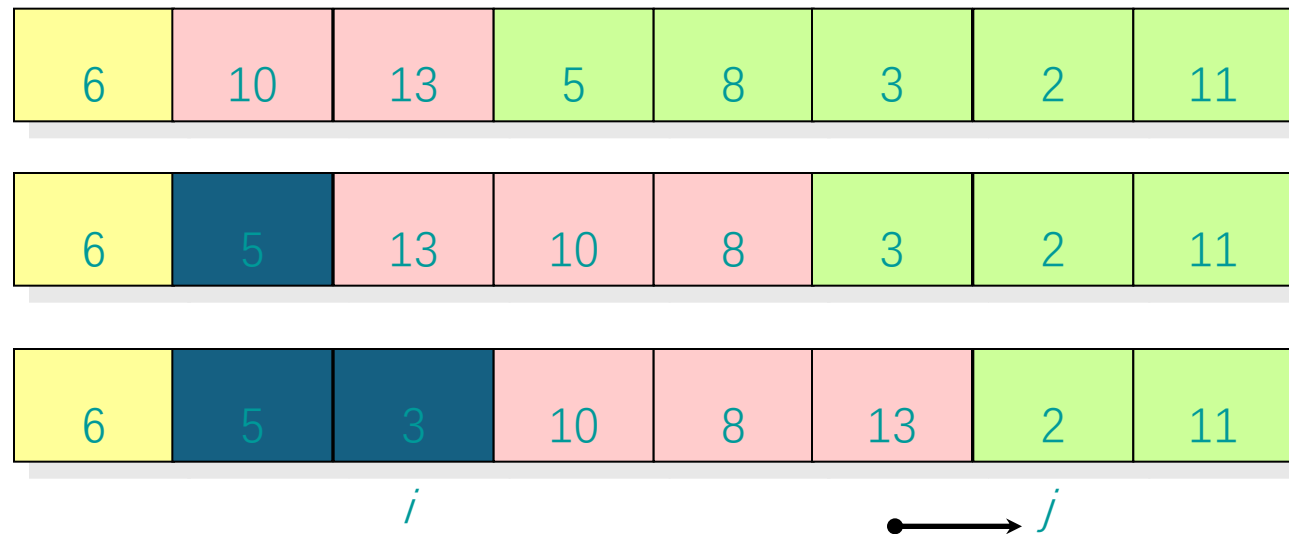
划分示例



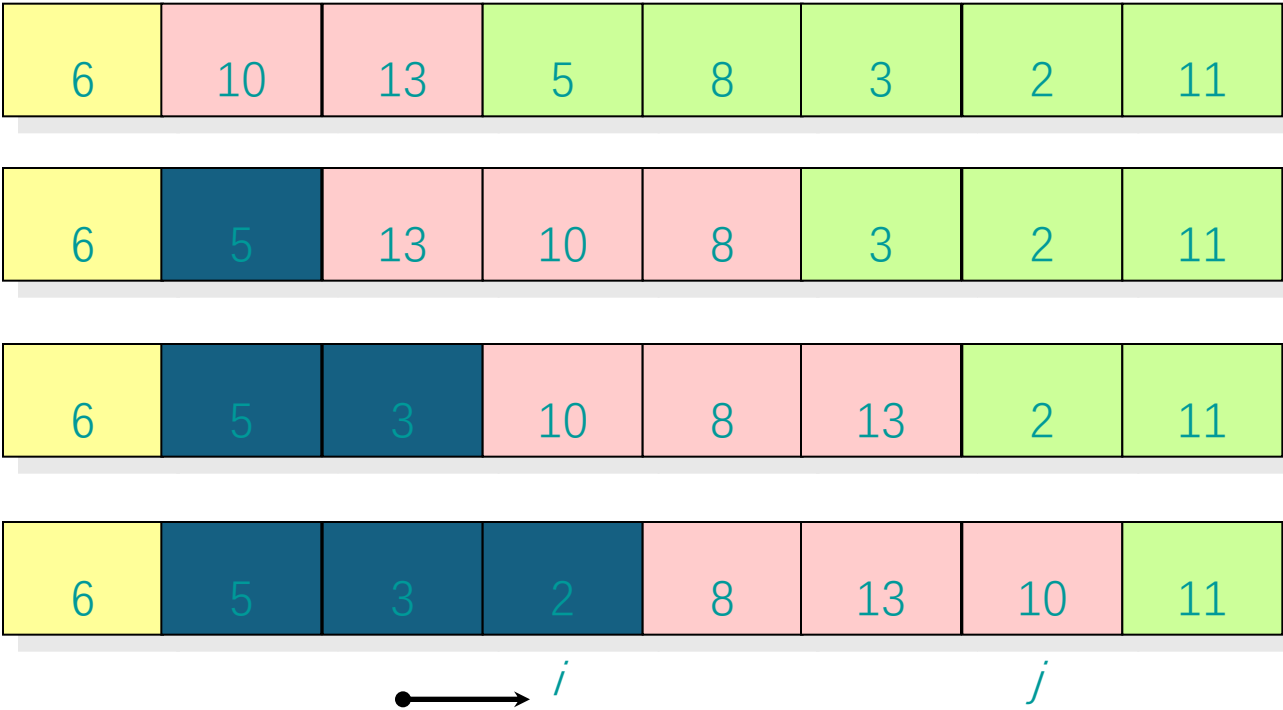
划分示例



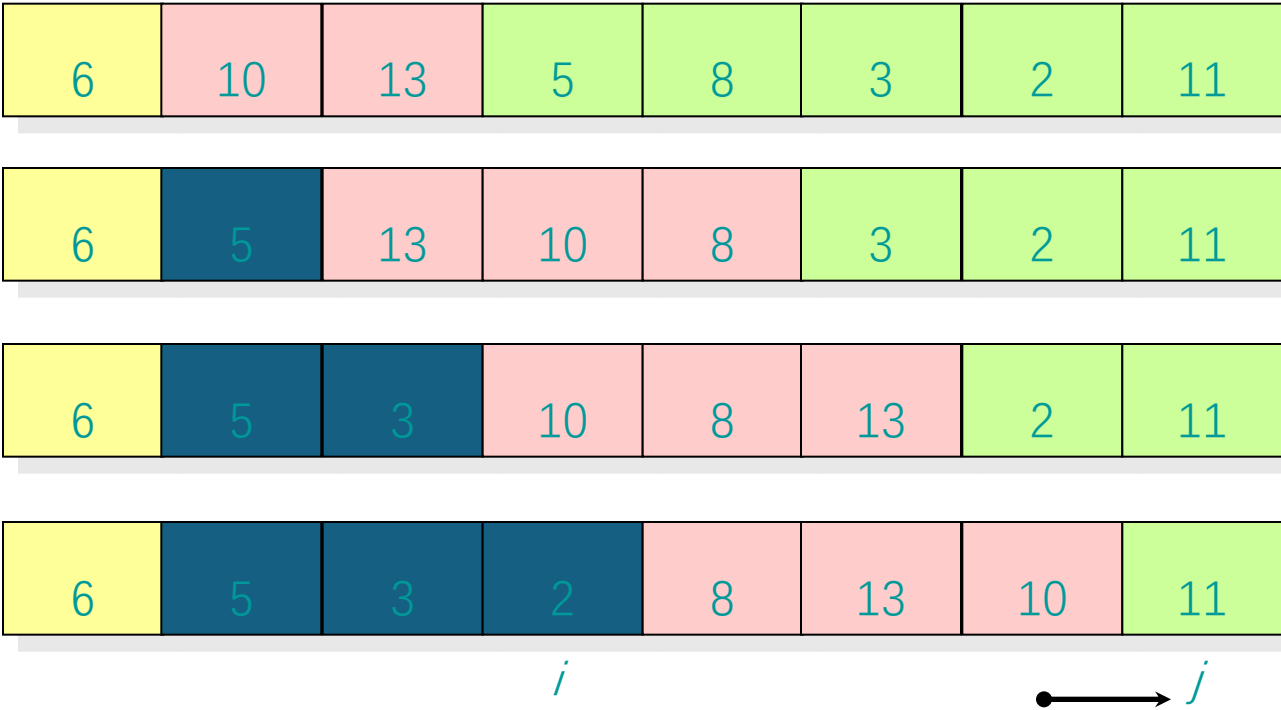
划分示例



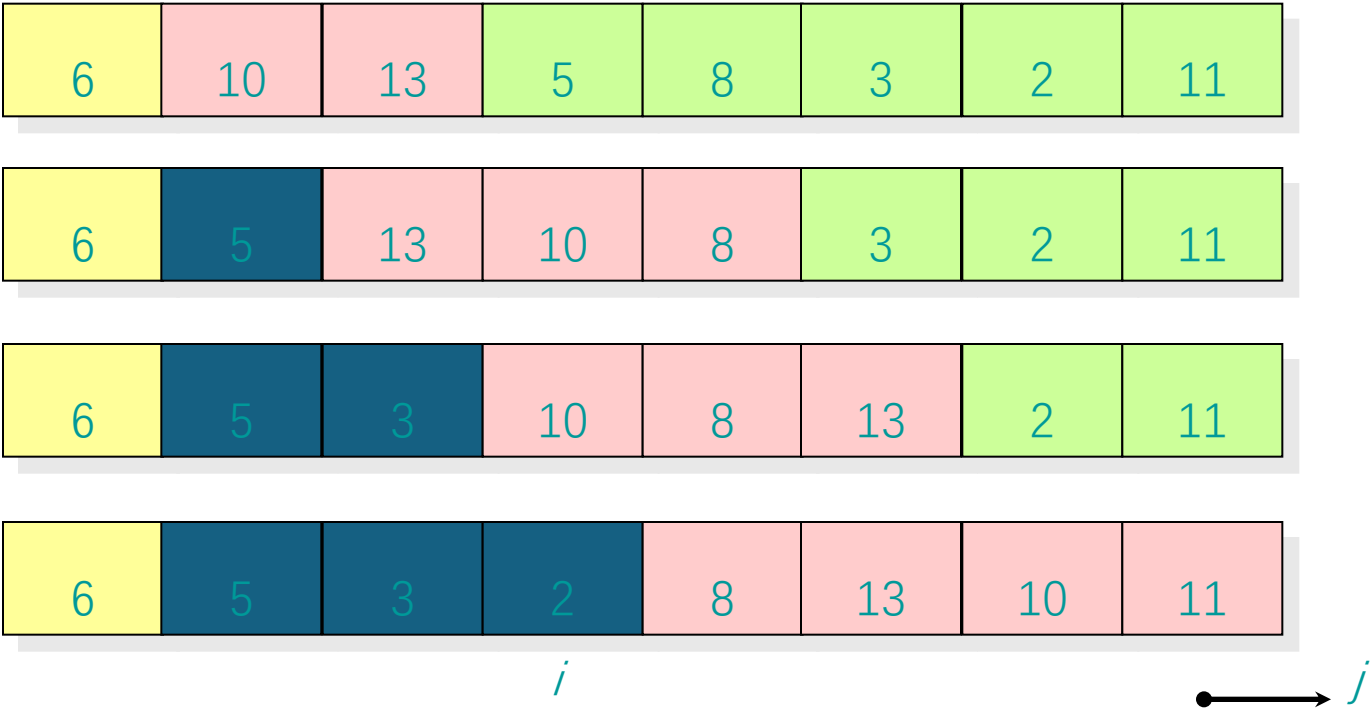
划分示例



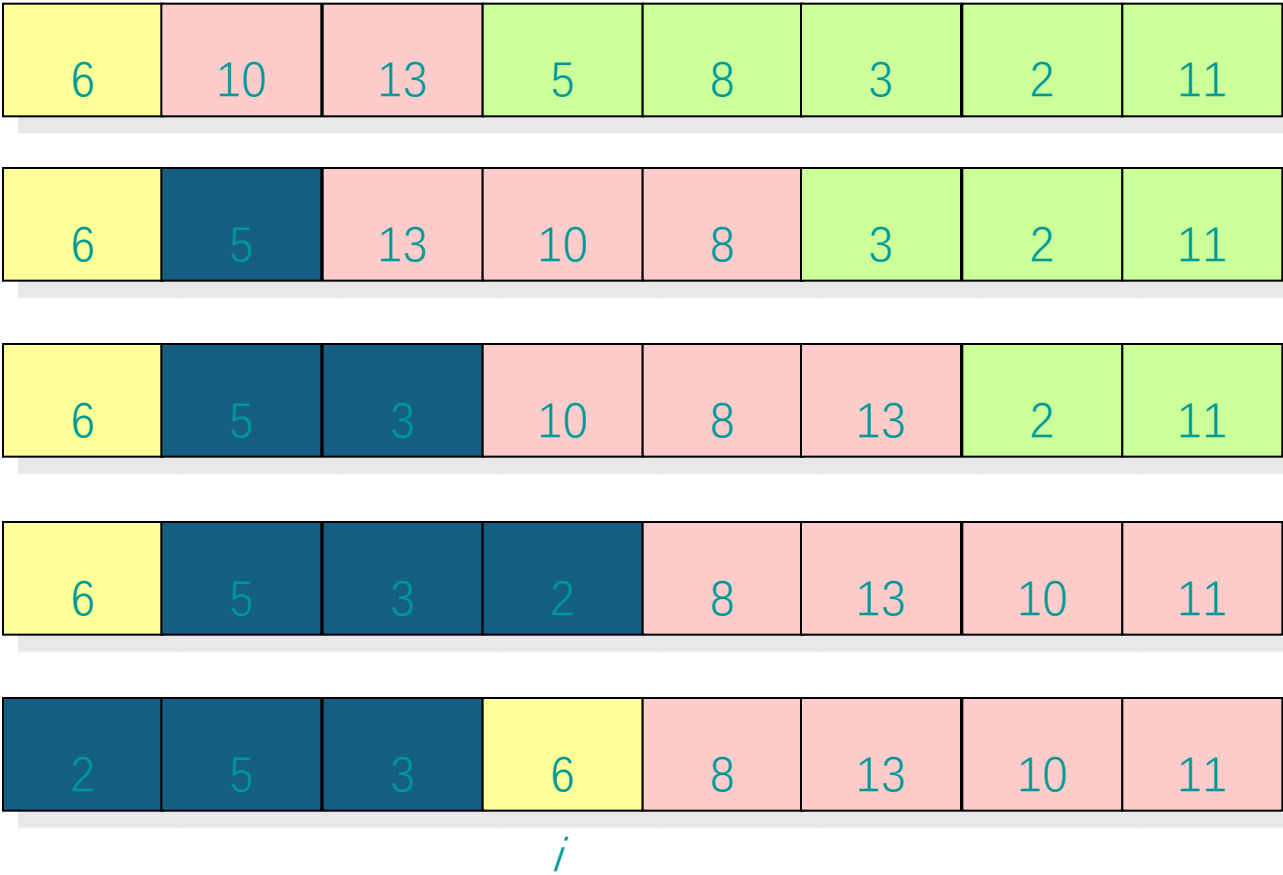
划分示例



划分示例



划分示例



Q7. 追踪两层

- Array:

[4, 1, 6, 2, 5, 3]

- **题目： 使用快速排序，以第一个元素为枢轴。 请给出：**
- 第一次划分后的数组
- 接下来递归处理哪些子数组

Q8. 完整快速排序追踪

- Array:

[4, 2, 5, 1, 3]

- **题目：** 使用快速排序，以第一个元素为枢轴。 给出每一步划分后的数组，直到排好序。

回顾：主定理

设 $f(n)$ 是对每个整数 $n > 0$ 都返回正值的函数。已知：

$$\begin{aligned} f(1) &= O(1) \\ f(n) &\leq \alpha \cdot f\left(\left\lceil \frac{n}{\beta} \right\rceil\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2) \end{aligned}$$

其中 $\alpha \geq 1, \beta > 1, \gamma \geq 0$, 且 $\lambda \geq 0$ 为常数。于是：

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

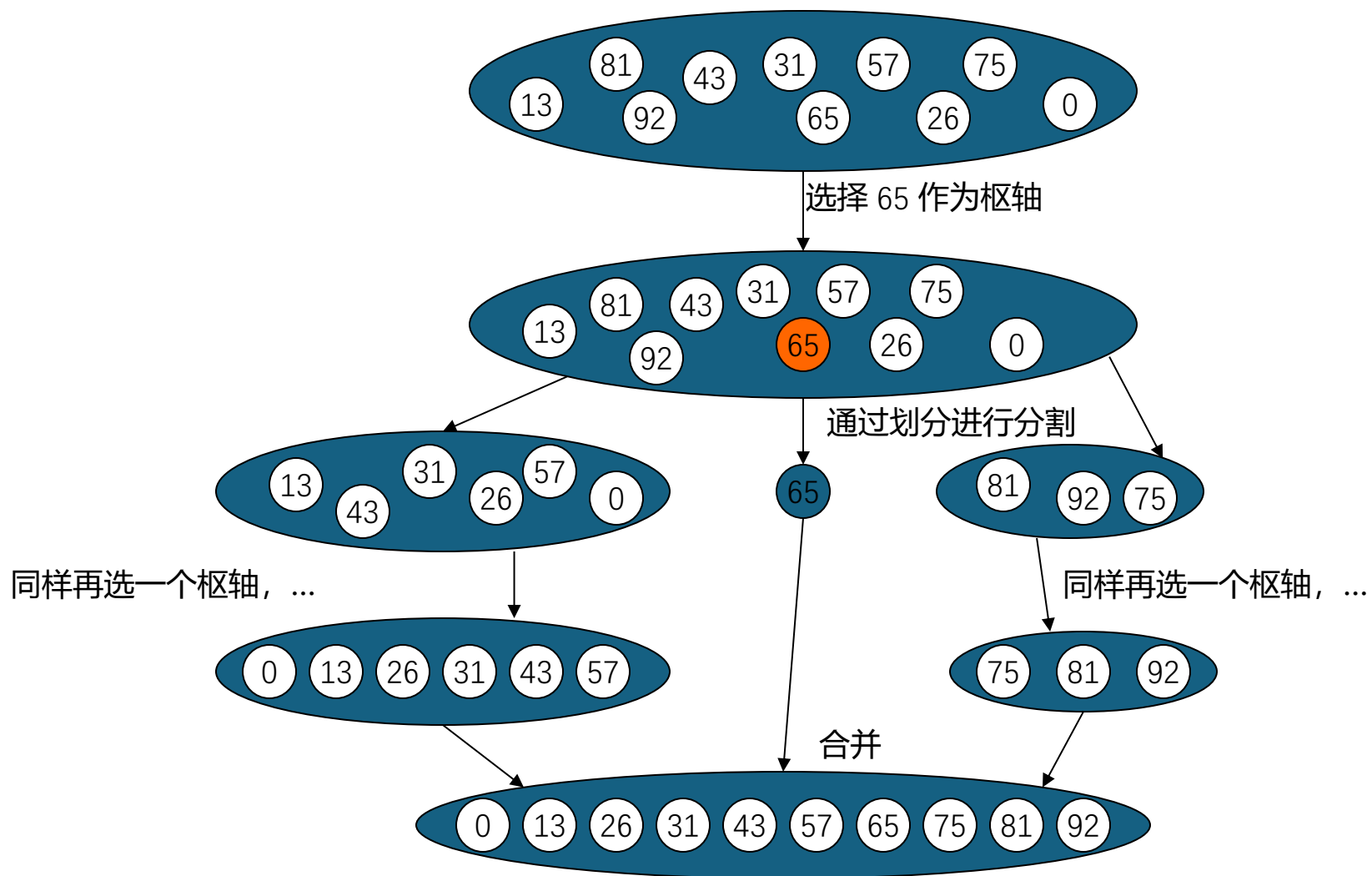
α : 每次将蛋糕分成的份数。

β : 与原蛋糕相比，每块大小缩小的倍数。

$g(n)$: 每次划分所需的额外工作量。

该定理可通过仔细应用前面见过的“展开法”来证明。细节繁琐，此处从略。

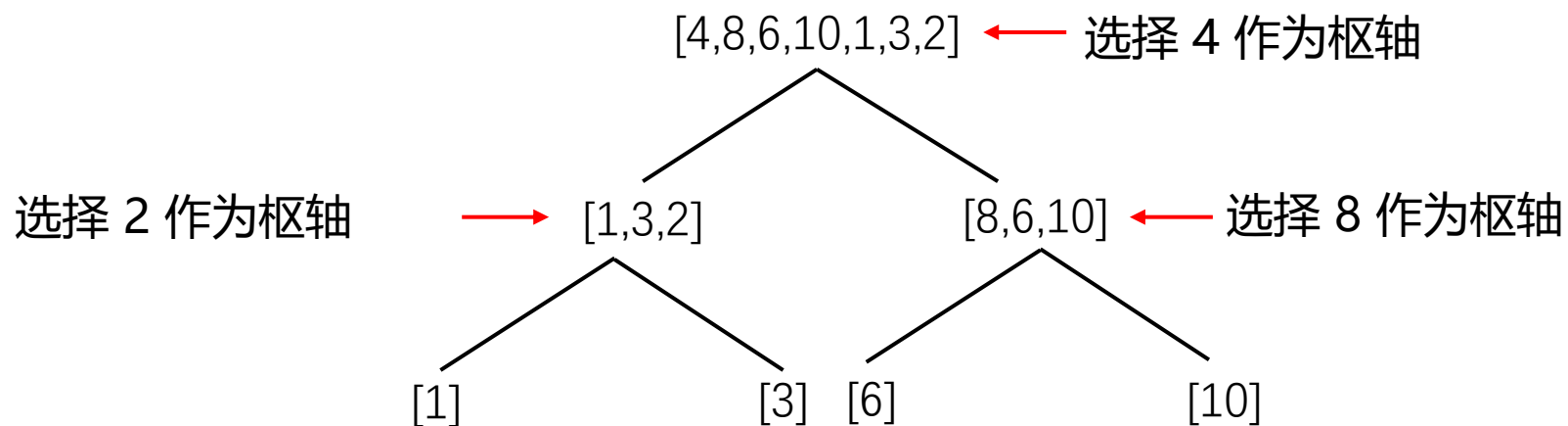
快速排序示例



最好情况分析

关键问题是如何选择枢轴。

- 若枢轴始终取中位数，则总时间可为 $O(n \log n)$ (n 个数的数组)。



Q9. 快速排序的最好情况

- 问题：
用主定理计算快速排序最好情况的时间复杂度

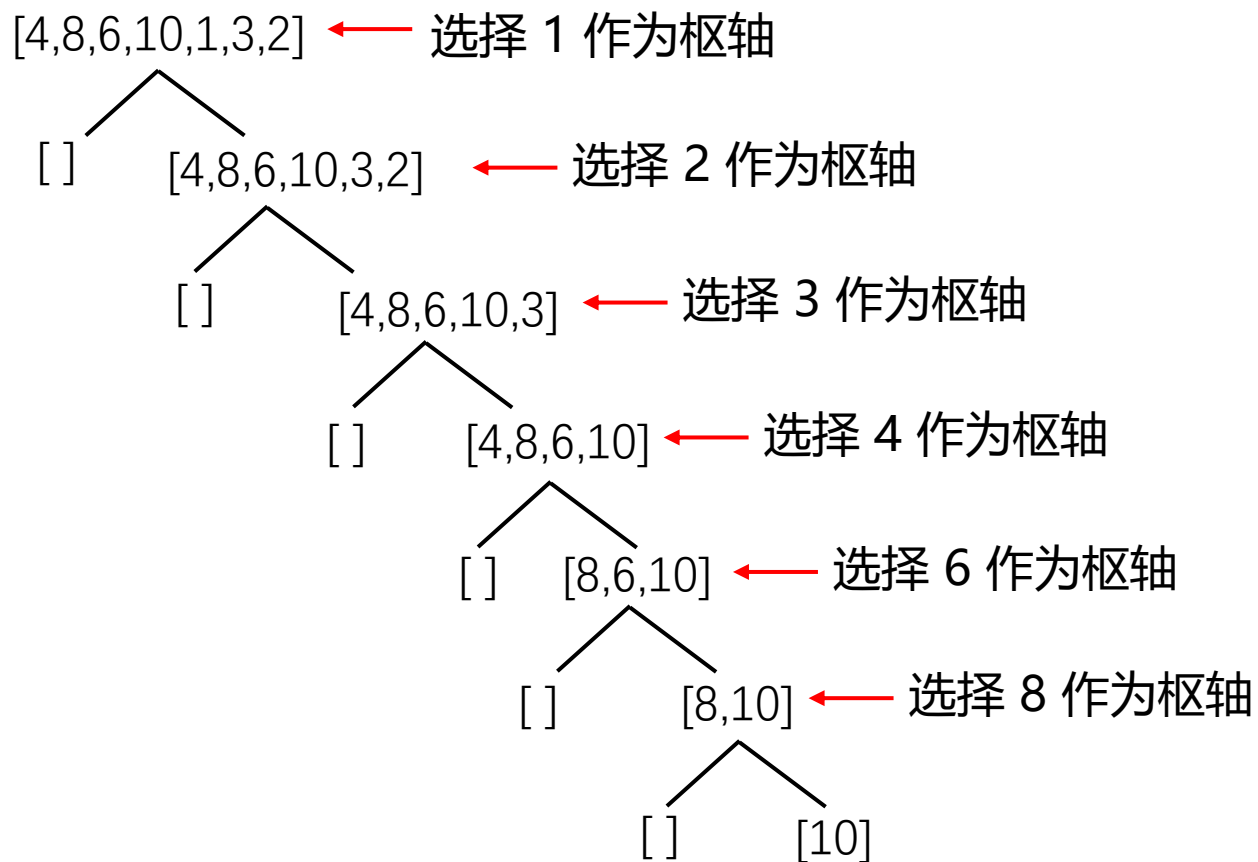
如何选择枢轴？

- 通常采用以下策略之一：
 - 总是选择第一个（或最后一个）元素作为枢轴。
 - **三数取中：取数组最左、最右和中间三个元素，选其中位数。**
 - **随机枢轴：从数组中随机选一个元素作为枢轴。**
- 这些策略的性能如何？

最坏情况分析

快速排序的最坏情况运行时间可能高达 $\Theta(n^2)$:

- 当枢轴始终取最小或最大的数时，就会出现这种情况。

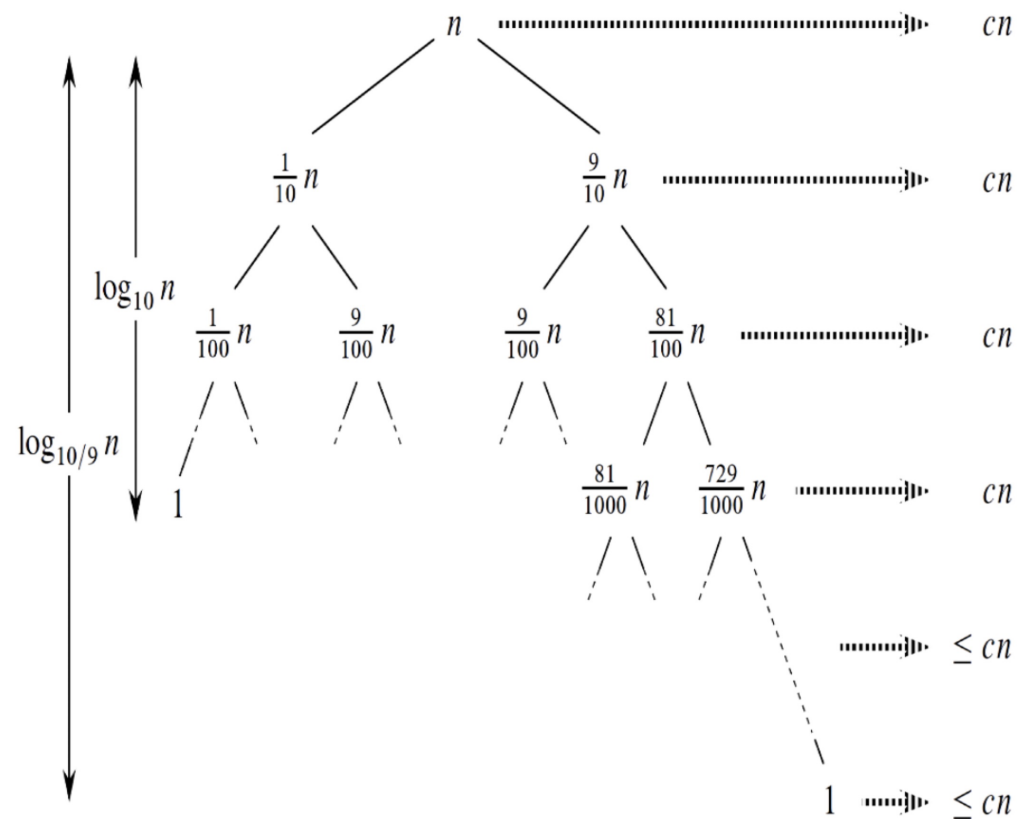


Q10. 快速排序的最坏情况

- **问题：**
用主定理计算快速排序最坏情况的时间复杂度

“接近最好” 情况的分析

- 考虑一个例子。假设每次选取枢轴后总是产生 9 比 1 的划分，即一侧为 9/10，另一侧为 1/10。
- 设 l 为最深递归层数。我们有
 - $\left(\frac{9}{10}\right)^l n \geq 1$, i.e., $l \leq \log_{\frac{10}{9}} n$.
- 因此，时间复杂度为 $O(n \log n)$ 。



$O(n \lg n)$

实践中的快速排序

- 随机化快速排序在许多输入上以 $O(n \log n)$ 时间运行。
 - 快速排序是一种实用的通用排序算法。
 - 快速排序通常比归并排序快一倍以上。
 - 快速排序可通过代码调优获得显著加速。
-
- <https://www.toptal.com/developers/sorting-algorithms>

大纲

- 主定理
- 快速排序
- **排序下界**

排序下界

- 我们已经知道, n 个元素可以在 $O(n \log n)$ 时间内排序。
- 本讲将证明该时间复杂度对基于比较的算法是最优的。换言之, 任何此类算法至少在某个输入上需要 $\Omega(n \log n)$ 时间。

排序下界

- 输入数组 A 中的 n 个元素共有 $n!$ 种不同的排列方式
- 示例：对于 $n = 3$, 6 种排列：

$A[1], A[2], A[3]$

$A[1], A[3], A[2]$

$A[2], A[1], A[3]$

$A[2], A[3], A[1]$

$A[3], A[1], A[2]$

$A[3], A[2], A[1]$

- 排序的本质目标是判定 $n!$ 种排列中哪一种是最终的有序顺序。

排序下界：基于比较的算法

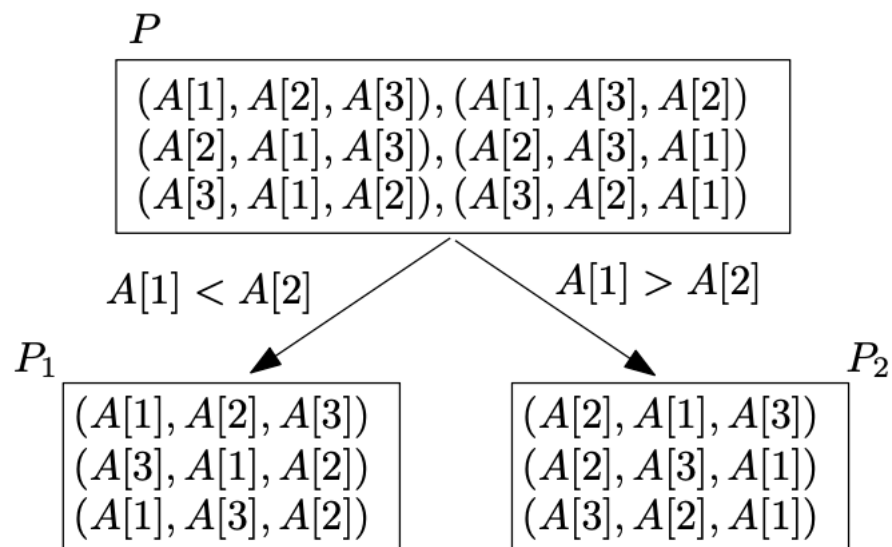
形式上，此类算法通过不断缩小可能排列的集合 P 来工作。

- 一开始， P 包含全部 $n!$ 种排列。
- 每次比较都使算法能够丢弃 P 中与比较结果不一致的所有排列。
- 最终， P 中只剩 1 种排列，它就是最终的有序顺序。

换言之，在任意时刻， P 中剩余的所有排列都是可能的结果。只要 $|P| \geq 2$ ，算法就不能终止。

排序下界

- 缩小候选集：一个例子



- 一般而言，每次比较可将 P 缩小为 P_1 或 P_2 。

排序下界：基于比较的算法框架

Framework

1. $P \leftarrow$ all the $n!$ permutations of A
2. **while** $|P| > 1$
3. make a comparison between elements e_1 and e_2
4. **if** $e_1 < e_2$ **then**
5. $P \leftarrow P_1$, where P_1 is the set of permutations in P
 consistent with $e_1 < e_2$
6. **else**
7. $P \leftarrow P_2$, where P_2 is the set of permutations in P
 consistent with $e_1 > e_2$
8. **return** the permutation in P

各类算法在如何实现第 3 步上有所不同。

排序下界：最坏情况的下界

形式上，此类算法通过不断缩小可能排列的集合 P 来工作。

- 注意， P_1 与 P_2 中必有一个至少包含 P 中一半的排列（即要么 $|P_1| \geq |P|/2$ ，要么 $|P_2| \geq |P|/2$ ）。
- 最坏情况发生在 P 总是缩小为 P_1 与 P_2 中较大的那个集合时。
- 此时，每次比较后 P 的规模至多减半。
- 因此，使 $|P|$ 减至 1 所需的比较次数为 $\log_2(n!)$ 。

下一页给出 $\log_2(n!) = \Omega(n \log n)$ 。

排序下界：最坏情况的下界

$$\begin{aligned}\log_2(n!) &= \sum_{i=1}^n \log_2 i \\ &\geq \sum_{i=n/2}^n \log_2 i \\ &\geq (n/2) \log_2(n/2) \\ &= \Omega(n \log n)\end{aligned}$$

由此得出，任何基于比较的算法在最坏情况下对 n 个元素排序都必须花费 $\Omega(n \log n)$ 时间。

带随机化的 RAM

回顾：RAM 模型的核心是一组原子操作。我们可以再增加一个原子操作：

- **RANDOM(x, y)**: 给定整数 x 和 y (满足 $x \leq y$) , 该操作返回 $[x, y]$ 中均匀随机选取的一个整数。
 - $x, x + 1, x + 2, \dots y$ 中每一个被返回的概率都相同

设 X 为等于某算法在某输入上代价的随机变量。该算法在该输入上的期望代价即为 X 的期望。

随机算法的最坏情况期望代价, 是该算法在所有规模为 n 的可能输入上的最大期望代价

带随机化的 RAM

示例：

- 问题 “Find-a-Zero”：设 A 为含 n 个整数的数组，其中一半为 0。设计算法报告 A 中任意一个含 0 的位置。
- 例如，设 $A = (9, 18, 0, 0, 15, 0, 33, 0)$ 。算法可报告 3、4、6 或 8。

一个随机算法：

1. **do**
 2. $r = \text{RANDOM}(1, n)$
 3. **until** $A[r] = 0$
 4. **return** r
- 该算法在任意输入 A 上都以期望 $O(1)$ 时间结束！

随机快速排序分析（不考）

输入：整数集合 S 以长度为 n 的数组 A 给出。

考虑快速排序的随机化版本：

- 从 A 中均匀随机选取整数 p 作为主元。

我们将证明，随机化快速排序在每个输入上的期望代价均为 $O(n \log n)$ 。

- 首先请确认：只需分析比较次数 X 即可。运行时间由 $O(n + X)$ 界定
- 接下来，我们将证明 $E(X) = O(n \log n)$ 。

随机快速排序分析

用 e_i 表示 S 中第 i 小的整数。对任意满足 $i \neq j$ 的 i, j , 考虑 e_i 、 e_j

快速排序比较 e_i 与 e_j 的概率是多少?

这个问题乍看很难, 答案却出人意料地简单。我们来观察:

- 每个元素恰好会被选为主元一次。
- 若介于二者之间的任一元素在 e_i 与 e_j 之前被选为主元, 则 e_i 与 e_j 不会被比较。

例如, 设 $i = 7$ 且 $j = 12$ 。若 e_9 为主元, 则 e_i 与 e_j 将被 e_9 分开 (思考: 为什么?), 此后算法中二者不再被比较。

随机快速排序分析

因此, e_i 与 e_j 被比较当且仅当二者之一是 $e_i, e_{i+1}, \dots, e_j$ 中最先被选为主元的。

该概率为 $2/(j - i + 1)$ 。

随机快速排序分析

定义随机变量 X_{ij} : 若 e_i 与 e_j 被比较则为 1; 否则 $X_{ij} = 0$ 。于是有 $\Pr[X_{ij} = 1] = 2/(j - i + 1)$. That is $E[X_{ij}] = 2/(j - i + 1)$.

显然, $X = \sum_{i,j} X_{ij}$ 。因此:

$$\begin{aligned} E[X] &= \sum_{i,j:i < j} E[X_{ij}] = \sum_{i,j:i < j} \frac{2}{j - i + 1} \\ &= 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j - i + 1} \\ &= 2 \sum_{i=1}^{n-1} O(\log(n - i + 1)) \\ &= 2 \sum_{i=1}^{n-1} O(\log n) = O(n \log n). \end{aligned}$$

随机快速排序分析

上述分析用到如下事实：

$$1 + 1/2 + 1/3 + 1/4 + \dots + 1/n = O(\log n).$$

左边称为调和级数。

练习 1 - 真实算法解读

- 某递归算法将规模为 n 的问题分成 4 个规模为 $n/2$ 的子问题，合并步骤代价为 n^2 。
 - 写出递推式
 - 用主定理求解

练习 2 - 划分结果

- 以最后一个元素为主元，对下列数组执行一次划分：

[7, 2, 9, 4, 3]

- 划分后的数组是什么？主元的最终下标是多少？

练习 3 — 快速排序最好情况递推式

- 追踪下列数组上的快速排序（取第一个元素为主元）：
- [5, 1, 4, 2, 8, 7, 6, 3]

练习 4 - 快速排序递归树形状

- 快速排序总是将数组划分为如下规模的子数组：

$$\frac{n}{4} \text{ and } \frac{3n}{4}$$

- 且划分代价为 $\Theta(n)$ 。
- 写出递推式
- 运行时间是否仍为 $\Theta(n \log n)$? 请直观解释